AIR UNIVERSITY, KAMRA

Department of Computer Science

FULL STACK WEB DEVELOPMENT

Assignment 05

AccessDesk

Role-Based Customer Access Portal using Node.js, Express.js, MySQL and jQuery AJAX

Student Name	Wasif Ali
Registration ID	245187
Class	BSCS-F-24-B
Course	Full Stack Web Development
Database / Table	mydb8 / customers4
Submission Deadline	27 October 2026

Table of Contents

1. Overview 3
1.1 Technologies 3
2. Roles and Access Rules 3
3. Access-Check Flow 3
4. Setup and Running 4
4.1 Requirements 4
4.2 Start the Application 4
5. API Routes 5
6. Security Design 5
6.1 Role Rules Enforced on the Server 5
6.2 Field Projection and Parameterized SQL 6
6.3 Summary of Safeguards 6
7. User Interface 6
8. Verification 6
8.1 Required Cases 7
8.2 Additional Cases 7
9. Screenshots 7
Screenshot 1: Roles 7
Screenshot 2: Search 9
Screenshot 3: 403 Forbidden 10
Screenshot 4: 429 Too Many Requests 10
Additional Screenshots 11
10. Assessment Checklist 13
11. Conclusion 13

1. Overview

AccessDesk is a web portal in which one customer database is shared by users with different levels of access. The caller is identified only by an API key sent in the x-api-key request header. The Node.js server decides what the caller may view or request, applies a row limit and a credit allowance, and returns only the fields that the role is allowed to see. The browser page (HTML, CSS and jQuery AJAX) sends requests and displays the answers; it contains no database credentials and makes no permission decisions.

1.1 Technologies

Layer	Technology
Frontend	HTML, CSS and jQuery (AJAX with JSON); the table updates without a page refresh
Backend	Node.js with Express.js, the cors middleware and the mysql package
Database	MySQL on XAMPP: existing database mydb8, table customers4 (columns name and address)
Requests	GET requests, the API key in the x-api-key header, JSON responses

2. Roles and Access Rules

Four API-key profiles are defined in server.js. The Student and Teacher limits of the lecture are unchanged; Guest and Admin were added. Each role has its own demo key, row limit, credit allowance, permitted actions and visible fields.

Role	Demo API key	Max rows	Credits	Permitted actions	Visible fields
Guest	`guest-key-000`	2	4	Load only	Name
Student	`student-key-123`	3	8	Load + Search	Name, Address
Teacher	`teacher-key-456`	10	30	Load + Search	Name, Address
Admin	`admin-key-789`	20	50	Load + Search + Usage	Name, Address

One approved API request uses one credit. When a role has no credits left, the next request is rejected with HTTP 429. Rejected requests (401, 403 and 400) do not use a credit. Credits are kept in server memory, so restarting Node.js resets them, as allowed by the assignment.

3. Access-Check Flow

Every customer route runs the same middleware chain before any SQL is executed. The checks are made in a fixed order, and the first failed check ends the request with its own status code.

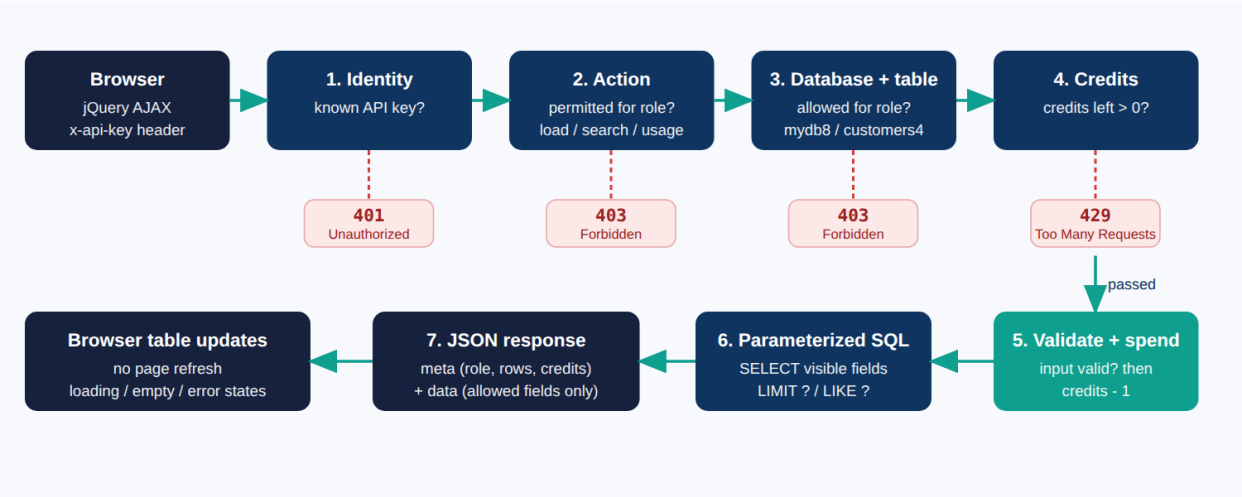


Figure 1: Order of access checks and request flow

1. **Identity (401).** The key is looked up with Object.hasOwn(), so names such as __proto__ cannot match. An unknown or missing key returns 401 and no data.
2. **Permitted action (403).** Each profile lists its allowedActions. A Guest calling Search, or a non-admin calling Usage, receives 403.
3. **Database and table (403).** The profile must list mydb8 and customers4 in allowedDatabases and allowedTables.
4. **Credits (429).** A role with zero credits is rejected before SQL runs.
5. **Validate and spend.** The search text is validated first (400 if empty or too long). Only then is one credit deducted.
6. **Parameterized SQL.** The row limit and the search pattern are bound values. The selected columns come from a fixed whitelist for the role.

4. Setup and Running

4.1 Requirements

- XAMPP with MySQL started. The existing database mydb8 with table customers4 (columns name and address) is used; no new SQL needs to be imported.
- If the table does not exist, import database/setup_optional.sql in phpMyAdmin. It creates the table and adds sample rows only when the table is empty.
- Node.js installed.

4.2 Start the Application

1. Open the accessdesk folder in VS Code and open a terminal in that folder.
2. Run `npm install` (first time only) and then `node server.js`. The terminal lists the four roles and prints "MySQL connected".
3. Open `http://localhost:3000` in the browser.
4. Alternative: copy `public/index.html` and `public/jquery.min.js` to `C:\xampp\htdocs\accessdesk\` and open `http://localhost/accessdesk/index.html`. CORS allows this.

Note. Restart Node.js before the verification sequence so that every role starts with its full credits. Open the page through http://localhost, not by double-clicking the HTML file, because a file:// page has no valid origin for CORS.

5. API Routes

Endpoint	Allowed roles	Behaviour
`GET /api/customers?limit=10`	All four roles	Returns min(requestedRows, maxRows) records with only the role's visible fields
`GET /api/customers/search?name=Ali`	Student, Teacher, Admin	Searches names with a bound LIKE value, limited to the role's maxRows. Guest receives 403
`GET /api/access/summary`	Admin only	Returns total, used and remaining credits for each role. Never returns API keys or database credentials

Successful customer routes return data together with metadata. A Guest response contains the name field only; the address column is not selected from MySQL and is never sent to the browser.

Example response of GET /api/customers?limit=10 for a Student

```
{
  "meta": {
    "role": "student", "remainingCredits": 7,
    "database": "mydb8", "table": "customers4",
    "fields": ["name", "address"],
    "requestedRows": 10, "allowedRows": 3, "returnedRows": 3
  },
  "data": [ { "name": "Ali Raza", "address": "Taxila" }, ... ]
}
```

6. Security Design

6.1 Role Rules Enforced on the Server

server.js: authorize() middleware

```
function authorize(action) {
  return function (req, res, next) {
    const key = req.get('x-api-key');
    const user = key && Object.hasOwn(USERS, key) ? USERS[key] : null;
    if (!user) return denied(res, 401, 'Unauthorized: invalid or missing API key');
    if (!user.allowedActions.includes(action))
      return denied(res, 403, 'Forbidden: ' + action + ' is not permitted ...', user);
    if (!user.allowedDatabases.includes(DATABASE)) return denied(res, 403, ...);
    if (!user.allowedTables.includes(TABLE))       return denied(res, 403, ...);
    if (user.credits <= 0) return denied(res, 429, 'Too Many Requests ...', user);
    req.currentUser = user;
    next();
  };
}
```

6.2 Field Projection and Parameterized SQL

server.js: role-specific columns and the bound search value

```
const COLUMNS = { name: 'name', address: 'address' };   // fixed whitelist
function selectSql(user) {
  const cols = user.visibleFields.map(f => COLUMNS[f]);
  return 'SELECT ' + cols.join(', ') + ' FROM ' + TABLE;   // guest: SELECT name FROM customers4
}

app.get('/api/customers/search', authorize('search'), validateSearch, spendCredit, function (req,
res) {
  const pattern = '%' + escapeLike(req.searchName) + '%';
  db.query(selectSql(user) + ' WHERE name LIKE ? LIMIT ?', [pattern, user.maxRows], ...);
});
```

6.3 Summary of Safeguards

- The Guest SQL statement selects only the name column, so the address never leaves the database for that role. Hiding it with CSS would not be secure.
- The row limit and the search pattern are passed as bound values; characters such as % and _ typed by the user are escaped so they are searched literally.
- MySQL credentials exist only in server.js. The browser page contains the demo keys and the API address only.
- The usage endpoint returns role names and credit counts only. API keys and database settings are never included.
- CORS accepts only localhost and 127.0.0.1 origins, allows the x-api-key header and answers the browser’s OPTIONS preflight. Other origins receive no CORS header and are blocked by the browser.
- If MySQL fails after a credit was deducted, the credit is refunded and a clear 500 message is returned.
- Unknown API endpoints return a JSON 404.

7. User Interface

The page follows the suggested layout: a role and API-key selector, a requested-rows field, a search field and three action buttons (Load Customers, Search Customers, View Usage). Three cards show the active role badge, the row allowance (for example "3 of 10 requested") and the credits left. The server answer is shown in a colour-coded status bar with the HTTP code, a loading indicator while a request is running, a results table that adapts to the fields returned, an empty-result message, and a request log of the latest calls. The Search and Usage buttons stay available for every role on purpose: the server, not the page, must be the one that refuses forbidden actions.

8. Verification

The cases below were executed against the running API after restarting Node.js. All of them produced the expected result.

8.1 Required Cases

No.	Test	Expected	Actual	Observed result
T1	Unknown API key	401	401 ✓	No customer records returned
T2	Guest loads 10 records	200	200 ✓	At most 2 names; no address field in the response
T3	Guest attempts Search	403	403 ✓	Search is not permitted for the guest role
T4	Student loads 10 records	200	200 ✓	At most 3 records
T5	Teacher loads 10 records	200	200 ✓	At most 10 records
T6	Student attempts a 9th request	429	429 ✓	No records returned; credits finished
T7	Non-admin opens usage endpoint	403	403 ✓	Access denied

8.2 Additional Cases

Test	Status	Observed result
Admin opens usage endpoint	200 ✓	roles: guest 3/4, student 0/8, teacher 29/30, admin 49/50; keys in response: False
Admin loads 10 -> max 20 allowed	200 ✓	10 rows (requested 10)
Admin requests 100 rows	200 ✓	20 rows (capped at maxRows 20)
Teacher searches 'Ali'	200 ✓	5 rows: Ali Raza, Alina Shah, Ali Hamza, Hina Malik, Shahzaib Ali
Search with empty name	400 ✓	Bad Request: enter a customer name to search
Search '%' is literal, not a wildcard	200 ✓	0 rows
SQL injection attempt in search	200 ✓	0 rows (treated as plain text)
Invalid limit 'abc' -> role maximum	200 ✓	10 rows
Unknown endpoint	404 ✓	API endpoint not found

9. Screenshots

Screenshot 1: Roles

Each role loads with 10 rows requested (the Admin requests 20). The row allowance card shows the role maximum, and the Guest table contains the Name column only.

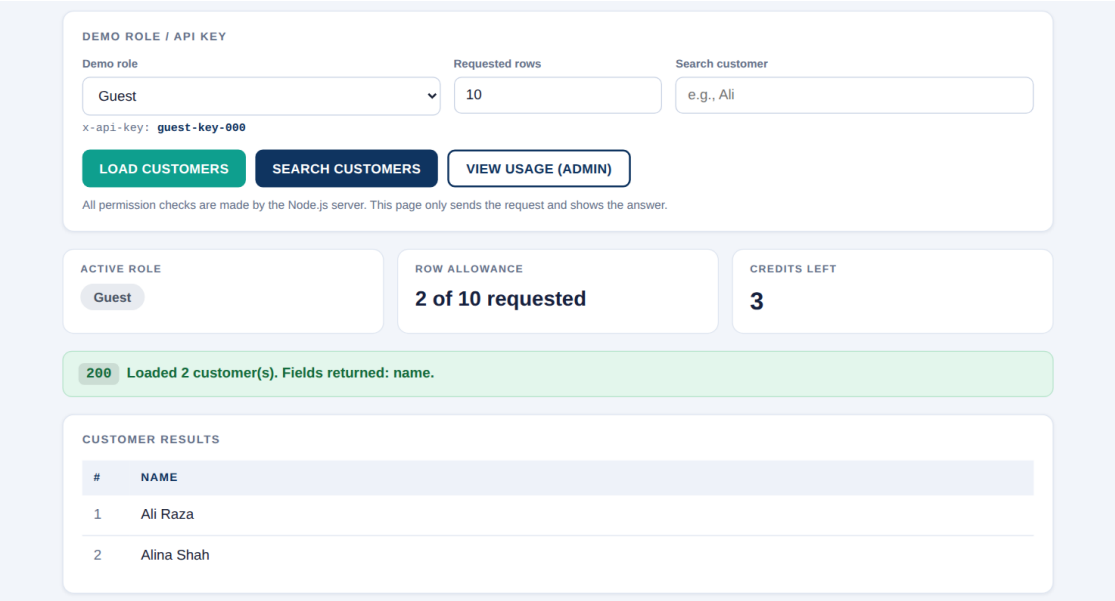


Figure 2: Guest: 2 of 10 rows, Name only, 3 credits left

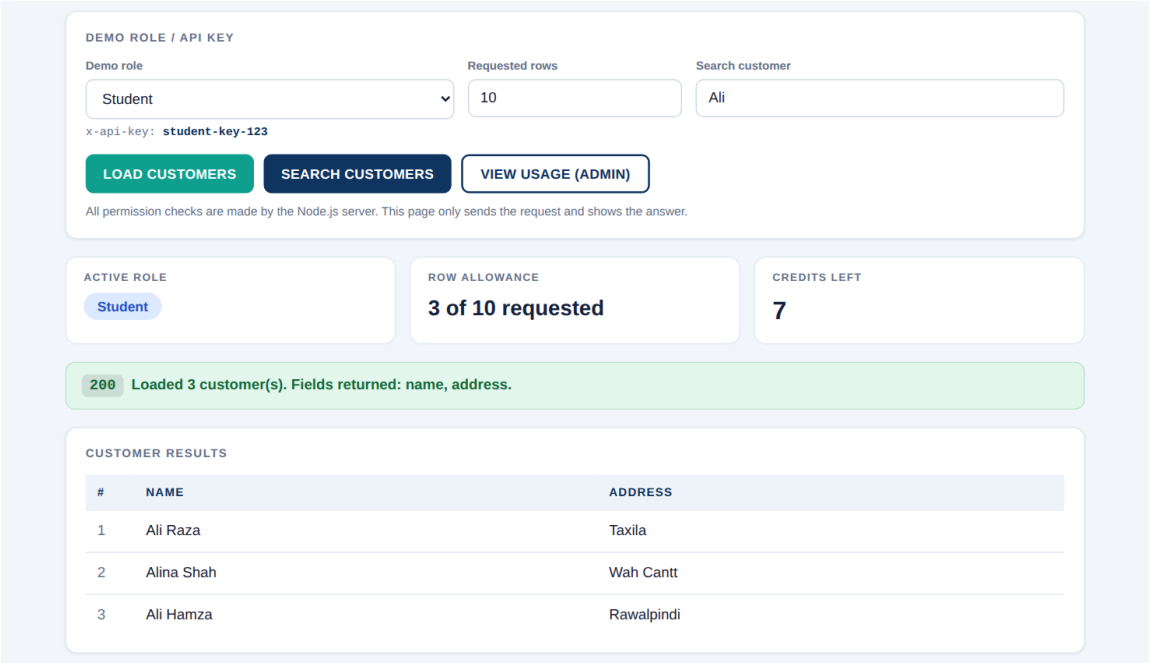


Figure 3: Student: 3 of 10 rows, Name and Address, 7 credits left

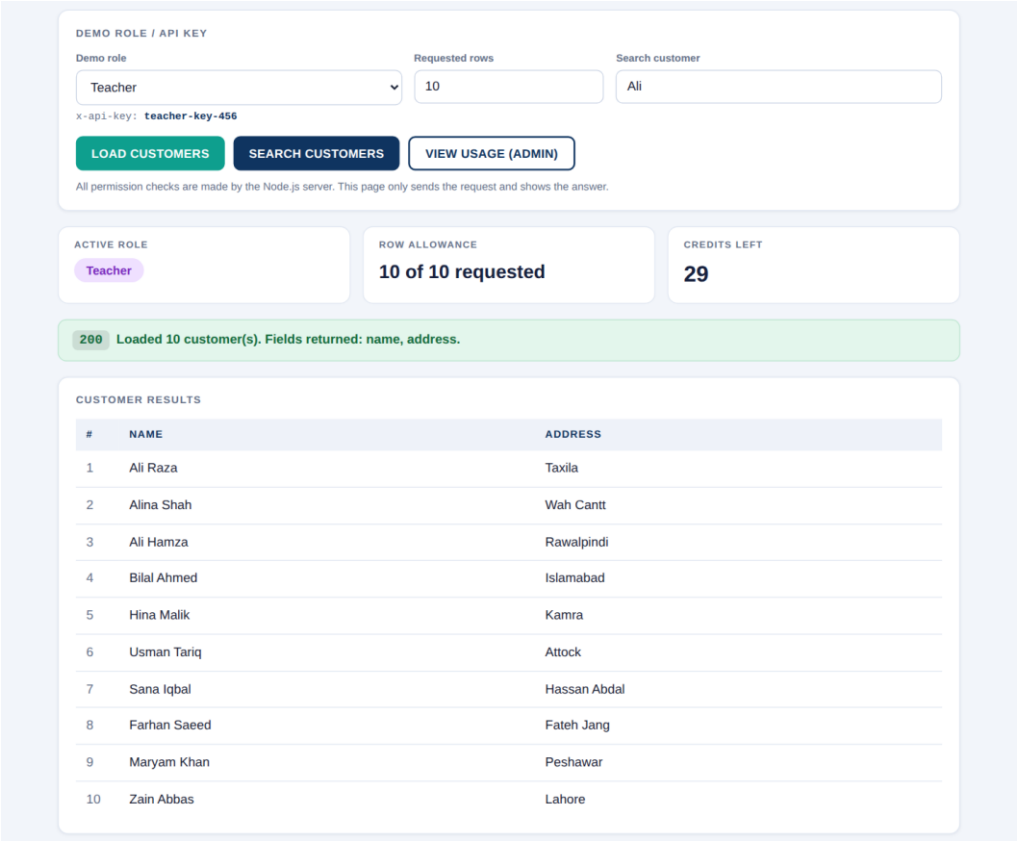


Figure 4: Teacher: 10 of 10 rows, 29 credits left

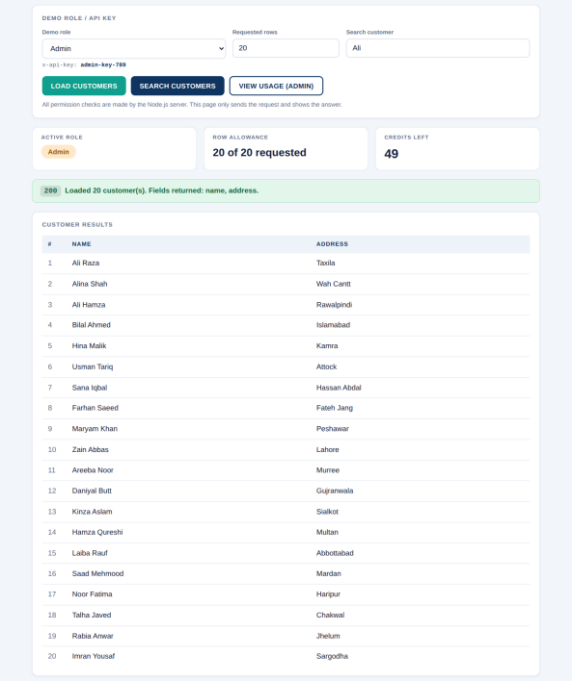


Figure 5: Admin: 20 rows allowed, 49 credits left

Screenshot 2: Search

A Student searches for "Ali". The result is limited to the Student maximum of 3 rows.

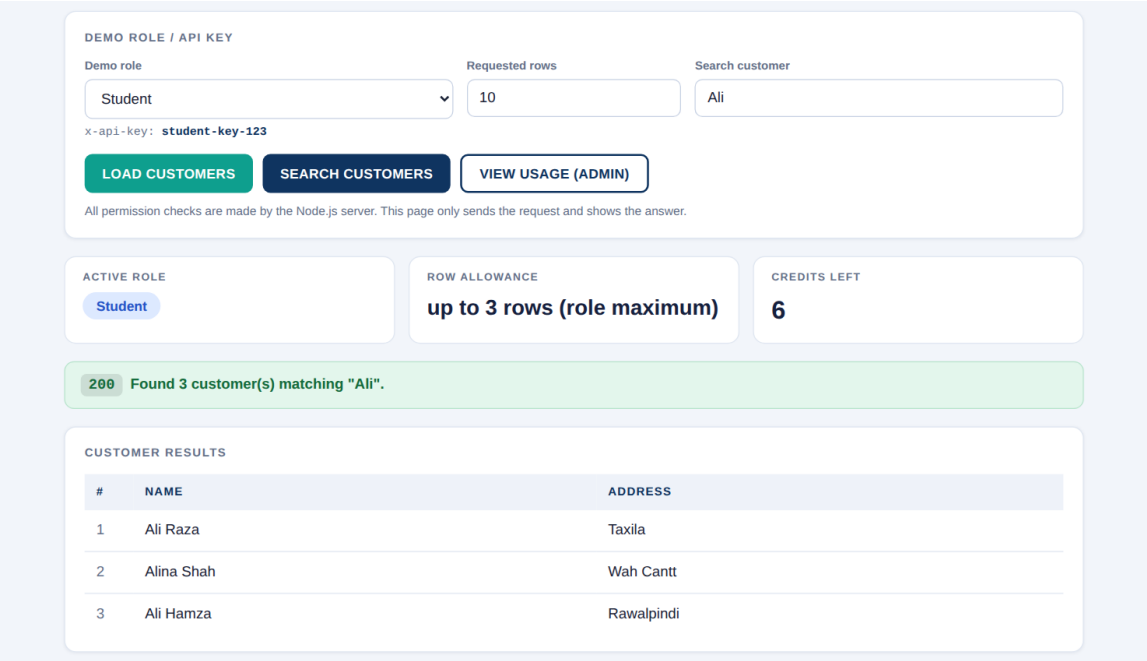


Figure 6: Search Customers as Student

Screenshot 3: 403 Forbidden

A Guest presses Search Customers. The server refuses the action and no credit is used.

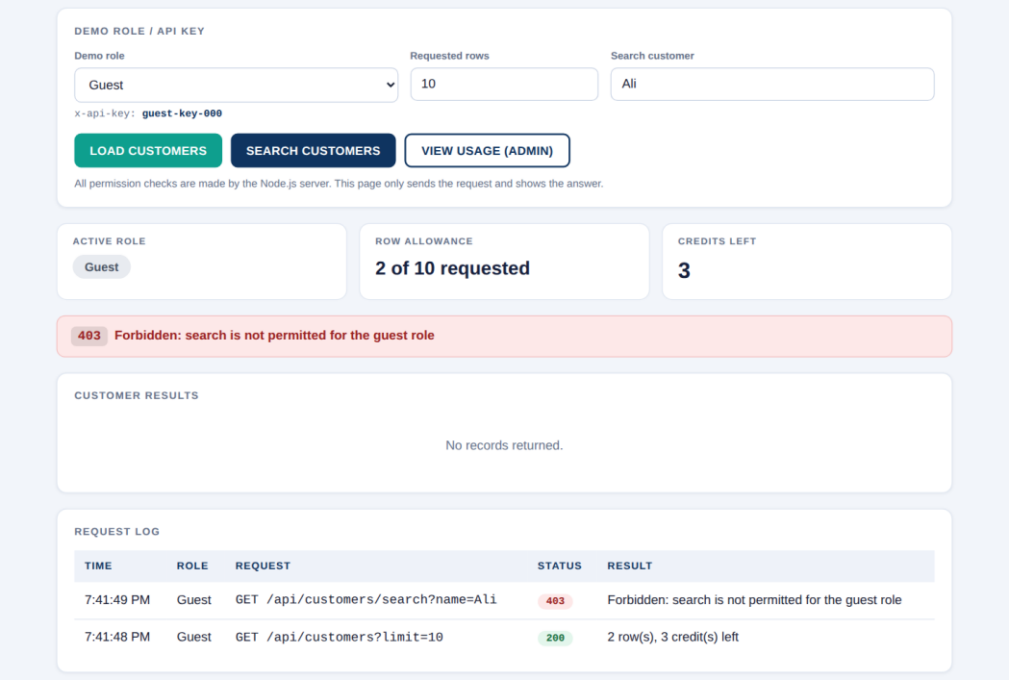


Figure 7: Guest attempts Search: HTTP 403

Screenshot 4: 429 Too Many Requests

The Student has used all 8 credits. The 9th request is rejected and no records are returned.

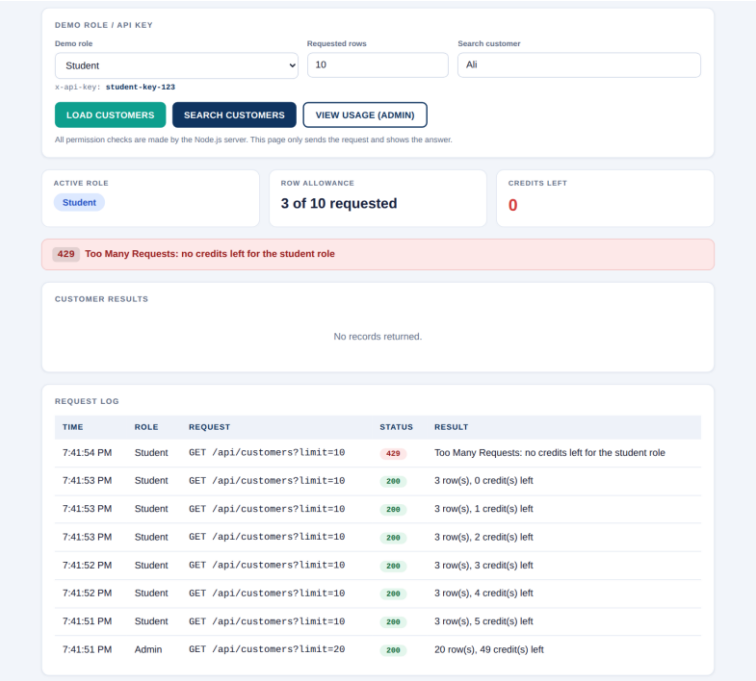


Figure 8: Student 9th request: HTTP 429

Additional Screenshots

Admin usage summary (credits per role, no API keys), unknown key (401) and a non-admin opening the usage endpoint (403).

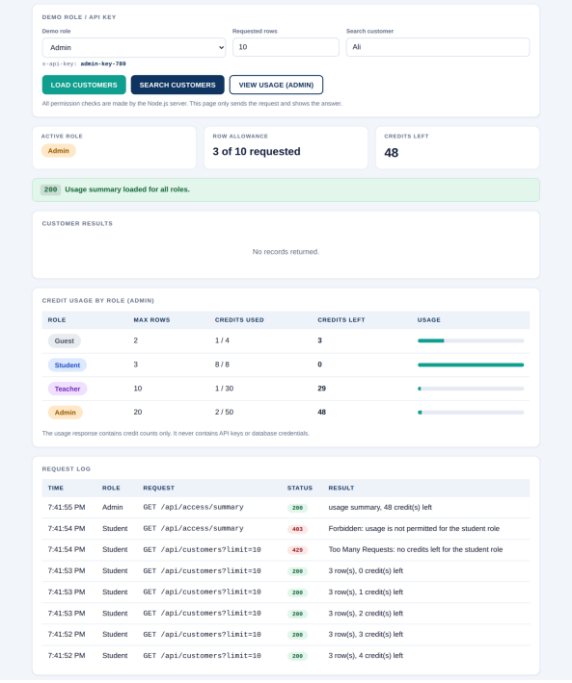


Figure 9: Admin: View Usage

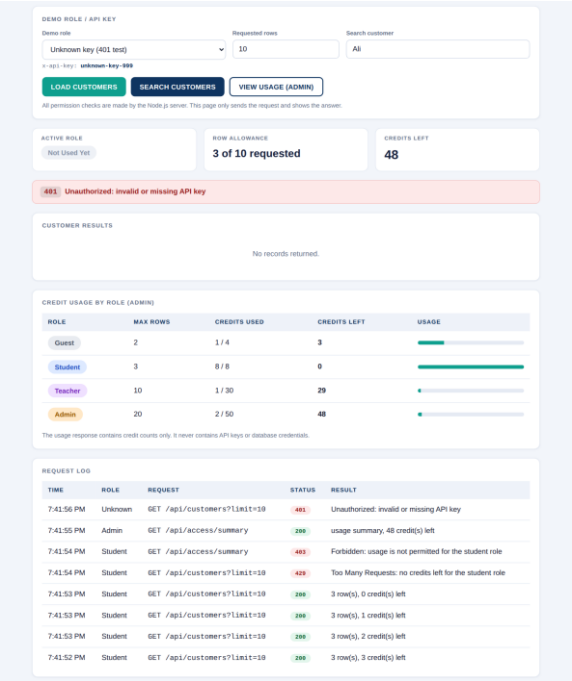


Figure 10: Unknown API key: HTTP 401

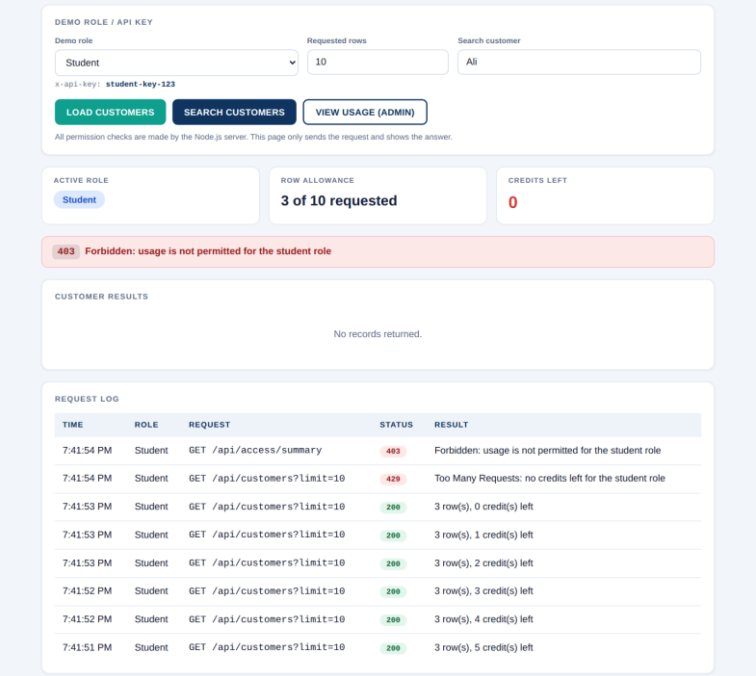


Figure 11: Student opens usage: HTTP 403, even with zero credits left

10. Assessment Checklist

Marking criterion	Marks	Where it is met
Role rules and secure checks	10	Four profiles, ordered checks 401/403/429, role-specific SQL columns, bound parameters (Sections 2, 3 and 6)
Load / Search / Usage routes	8	Three routes with min(requestedRows, maxRows), role maxRows for search and admin-only usage (Section 5)
Frontend presentation	6	Role badges, loading state, empty message, credits, status messages and request log, no page refresh (Section 7)
Testing and submission quality	6	All verification cases, four screenshots, README, optional SQL file and this report (Sections 8 and 9)

11. Conclusion

AccessDesk shows that access control must be enforced on the server. The browser only presents the interface; Node.js identifies the caller from the API key, checks the permitted action, database, table and credits, limits the rows, selects only the allowed columns with parameterized SQL, and answers with clear HTTP status codes. Guests cannot search or see addresses, students and teachers have different limits, the admin can inspect credit usage without ever seeing keys, and exhausted credits stop further requests.